
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries that unifies autoregressive modeling with flow matching within a single coherent framework. Conditioning on historical measurements and, where available, future covariates, we cast forecasting as sampling from a learned conditional distribution over future trajectories and factorize this joint distribution autoregressively into a sequence of per-step conditional densities. A single shared flow models each density, transporting a simple base distribution to the distribution of the next observation conditioned on an encoding of the past context together with the observed covariates. Instantiating these transformations within the flow matching (FM) framework affords scalable, simulation-free learning of the underlying probability paths. Coupling the autoregressive factorization with the FM objective allows FLOWTIME to retain the defining strengths of autoregressive models—reliable extrapolation beyond the training horizon, a compact parameterization, and well-calibrated uncertainty—while simultaneously capturing the complex, multi-modal conditional distributions characteristic of modern transport-based generative models, thereby reconciling two model families whose complementary strengths had previously proven difficult to combine. Across both stochastic dynamical systems and a diverse suite of real-world forecasting benchmarks, FLOWTIME produces accurate, well-calibrated predictions and attains state-of-the-art or highly competitive performance, establishing autoregressive flow matching as an effective and practical approach to probabilistic time-series forecasting.

1 INTRODUCTION

Probabilistic timeseries forecasting—the task of inferring, from observational data, how the dynamics of a system evolve into the future together with a calibrated account of the attendant uncertainty—is a foundational problem in modern machine learning. It underpins decision-making across a broad spectrum of scientific, industrial, and societal domains (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024), in which the operative requirement is not merely accurate point predictions but faithful, well-calibrated predictive distributions over entire future trajectories.

A prominent and rapidly advancing line of work addresses this problem with deep generative models (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021), recasting forecasting as sampling from a future probability density conditioned on the past. Diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020), in particular, have advanced the state of the art across numerous forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). This expressiveness, however, comes at a substantial computational cost: because they rely on long, iterative stochastic denoising chains, diffusion models are typically expensive both to train and to sample from.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a principled route around this bottleneck. FM subsumes diffusion models as a special case within a single unifying framework while affording substantially greater latitude in the design of the generative process. In place of iterated stochastic denoising steps over a discretized trajectory, FM regresses a deterministic velocity field that induces a probability path transporting an arbitrary base distribution directly onto the target distribution through a continuous normalizing flow. Because these proba-

bility paths can be prescribed explicitly—for instance, as straight-line interpolants—FM dispenses with the handcrafted noise schedules and lengthy ancestral sampling chains intrinsic to diffusion, yielding markedly more efficient training and sampling.

FM has recently been brought to bear on timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024), delivering strong empirical results together with the improved computational efficiency inherited from the framework. Existing formulations, however, share a common design choice that entails a consequential limitation: they learn, in a single shot, the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. Convenient though it is for fast training and sampling, this monolithic treatment of the forecast horizon extrapolates poorly beyond the training distribution and yields miscalibrated uncertainty estimates. It further compounds the optimization burden, forcing the model to fit intricate, time-dependent probability paths jointly across the entire flow dimension. This last difficulty is made explicit by (Kollovieh et al., 2024), who introduce Gaussian-process-informed priors to simplify and accelerate optimization—an effective remedy that nonetheless confines their method to the univariate setting.

We propose FLOWTIME, a simple yet powerful method that harnesses FM to learn probabilistic forecasting models scaling natively to high-dimensional multivariate data without recourse to informed priors. Its central departure from existing FM-based methods, which model the whole future window at once, is a principled return to an autoregressive factorization: FLOWTIME decomposes forecasting into a sequence of per-step conditional distributions and models the distribution of each future time step, given past observations and covariates, through a single shared flow that transports a simple base distribution to the distribution of the next observation. This design directly reconciles two families of models whose complementary strengths had hitherto proven difficult to combine. By restoring the autoregressive structure, FLOWTIME accommodates variable forecasting horizons, extrapolates reliably beyond the training distribution, and furnishes well-calibrated uncertainty estimates; by situating that structure within flow matching, it replaces a single hard, high-dimensional optimization problem with a sequence of markedly simpler ones while preserving in full the multimodal expressiveness of transport-based generative models.

We make three principal contributions. (i) We introduce *autoregressive flow matching*, a factorization of the forecasting distribution that models each future step with a single shared, covariate- and context-conditioned flow and admits fully parallel, teacher-forced training. (ii) We establish that this factorization simplifies the learning problem and, unlike prior FM-based forecasters, scales natively to high-dimensional multivariate targets without recourse to informed priors. (iii) Through controlled ablations on stochastic dynamical systems and evaluations on established real-world benchmarks, we demonstrate that FLOWTIME delivers accurate forecasts, sharply improved extrapolation, and well-calibrated uncertainty, thereby furnishing a practical and effective solution to probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling paradigm that enables the training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at a scale previously out of reach. CNFs represent data transformations as solutions of ordinary differential equations (ODEs), yielding invertible mappings that admit tractable likelihood evaluation.

In common with CNFs, FM seeks a time-dependent diffeomorphic map on the data space Ω , termed a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transports a sample $\mathcal{X}^0 \sim p$ drawn from a source distribution p to a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ distributed according to a target distribution q , i.e. $\mathcal{X}^1 \sim q$. The flow is obtained as the solution of the initial value problem

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where the vector field $\mu: [0, 1] \times \Omega \rightarrow \Omega$ specifies the instantaneous velocity of the flow and generates a probability path $(p^s)^{0 \leq s \leq 1}$, each p^s being a distribution over Ω with $p^0 = p$. The learning problem is therefore to identify a vector field μ whose induced flow satisfies the terminal condition $p^1 = q$.

Whereas CNFs learn this vector field by likelihood maximization—which entails solving and differentiating through Eq. (1) at every training step—FM instead regresses a parametric vector field ν_θ

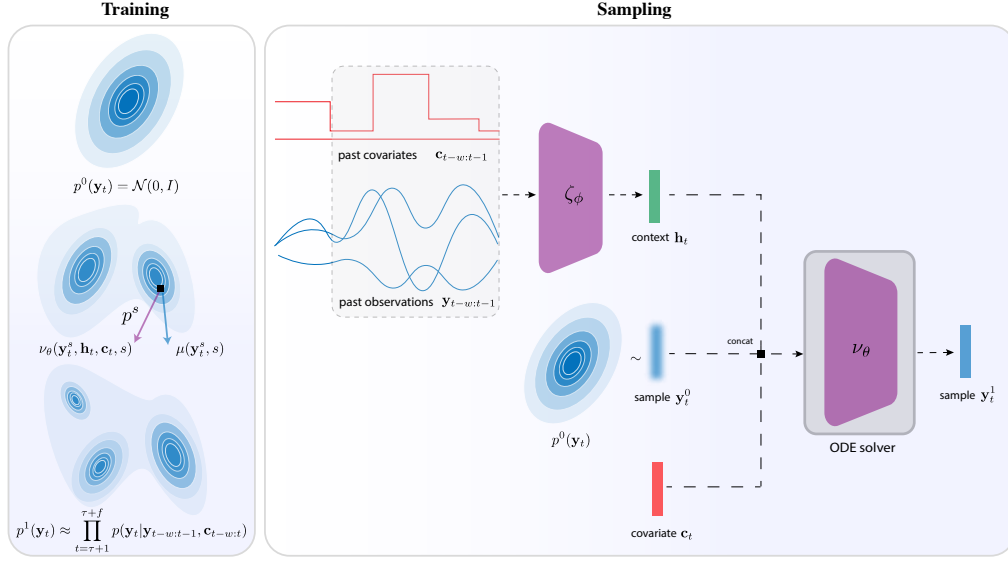


Figure 1: Overview of the FLOWTIME training and sampling procedures. During training, a velocity field μ generates a probability path $(p^s)_{0 \leq s \leq 1}$ that interpolates between a base distribution p^0 and the target distribution p^1 . Training regresses this field onto a neural network ν_θ conditioned on three quantities: a sample $\mathbf{y}_t^s$ drawn from the probability path at flow step s , a context vector $\mathbf{h}_t$ that encodes the past observations and covariates, and the current covariate $\mathbf{c}_t$. At inference time, a sample from the target distribution is obtained by drawing from the base distribution p^0 and integrating the learned velocity field with an ODE solver forward to $s = 1$.

with parameters θ directly onto the velocity fields of tractable, fixed conditional probability paths. This substitution yields the tractable conditional flow matching (CFM) objective,

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Once this objective is minimized, samples from q are generated by integrating Eq. (1) with the trained field ν_θ . Crucially, because the regression target is available in closed form, FM sidesteps backpropagation through the ODE solver altogether, rendering training stable and scalable even for high-dimensional generative tasks such as image and video synthesis (Esser et al., 2024; Polyak et al., 2024). A broad range of probability-path designs and conditioning strategies instantiate this template, unifying a wide family of transport-based generative models within one framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the l most recent measurements, inclusive of the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we seek probabilistic forecasts of the next f future values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, optionally conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, the objective is to learn to sample from the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ given a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ comprising m instances.

We factorize this conditional distribution autoregressively across future time steps under a Markov assumption of order w , obtaining

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ denotes the history size. The learning problem thus reduces to modeling the single per-step conditional in Eq. (3). Departing from the maximum-likelihood estimation conventionally used

to train autoregressive models, we learn this conditional within the flow matching framework (Lipman et al., 2022), thereby uniting the sequential inductive bias of autoregression with the expressive, multi-modal generative capacity of transport-based models within a single estimator.

This autoregressive factorization confers four concrete and mutually reinforcing advantages. First, it decomposes an otherwise intractable high-dimensional joint distribution into a sequence of substantially simpler per-step conditionals $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$, materially easing optimization. Second, because each conditional is trained under teacher forcing, all time steps are optimized in parallel rather than sequentially, so training cost does not scale with the forecast horizon. Third, the Markov order w furnishes an explicit and tunable trade-off between model capacity and computational cost. Finally, the factorization affords interpretable uncertainty quantification, since the chain of conditional distributions makes explicit how predictive uncertainty propagates forward in time.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau + 1, \tau + f]$, we specify a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with endpoint marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical data distribution. This path is generated by a flow $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ that evolves along the flow variable s according to the ordinary differential equation (ODE)

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the velocity field that induces the flow, chosen so that the resulting path satisfies the continuity equation.

To render the learning objective tractable, we express the marginal probability path as a mixture of conditional probability paths,

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where the conditioning variable $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is drawn from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 of (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional paths $p^s(\mathbf{y}_t | \mathbf{z})$ recover, in expectation, the otherwise intractable marginal velocity field $\mu(\mathbf{y}_t, s)$ —precisely the property that renders the marginal objective tractable and thereby licenses regression against the closed-form conditional targets below.

Following (Tong et al., 2023), we adopt the independent coupling $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are drawn from p^0 and p^1 , respectively, and define the conditional probability path as

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

whose marginals satisfy $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ in the limit $\sigma^2 \rightarrow 0$. The associated conditional velocity field then reduces to the particularly simple, time-independent form $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction yields straight-line probability paths between source and target in Euclidean space together with closed-form conditional velocities; it is precisely this simplicity that renders both training and sampling efficient. Once the velocity field is learned, samples from the target conditional $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by integrating the ODE in Eq. (4) forward to $s = 1$.

3.2 TRAINING

As in the standard FM setting, we learn this velocity field by regressing a neural network ν_θ with parameters θ against the conditional flow matching objective of Eq. 2. To condition the flow on the history and covariates, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t ,

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . The joint learning objective for the two networks then assumes the form

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

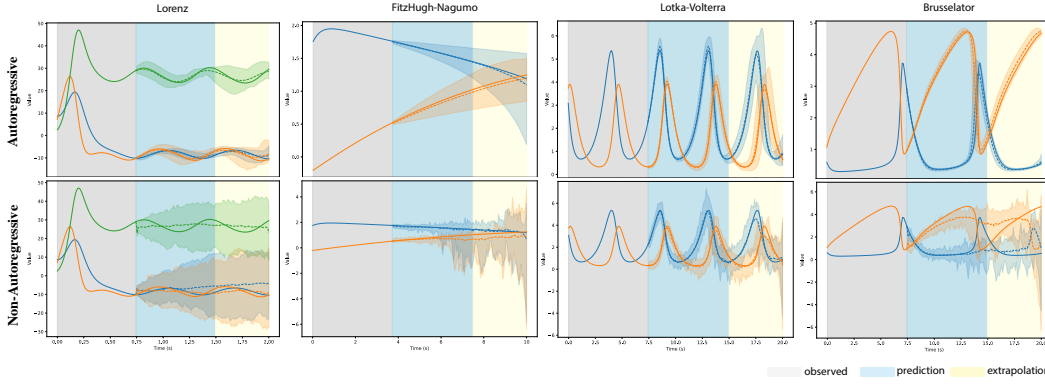


Figure 2: Forecast samples from autoregressive flow matching (FLOWTIME) versus standard, non-autoregressive flow matching on four dynamical systems. Solid lines denote the ground truth, dashed lines the mean prediction, and shaded bands the 95% confidence interval. Each panel spans both the prediction and the extrapolation regimes, and the contrast is unmistakable: whereas the non-autoregressive baseline degrades sharply once it leaves the training horizon, the autoregressive factorization sustains accurate, well-calibrated predictions far beyond it.

which we optimize by drawing an observation at a time point t , together with its context window and covariates, from $\mathcal{D}$ and jointly minimizing $\mathcal{L}(\theta, \phi)$ over ϕ and θ by stochastic gradient descent. This training scheme is an instance of teacher forcing (Williams & Zipser, 1989): ground-truth past observations, rather than the model’s own predictions, serve as context throughout training. Consequently, all time steps are optimized in parallel, and the error compounding that afflicts sequential rollout is eliminated at training time. Algorithm 1 sets out the complete autoregressive flow matching training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

3.3 INFERENCE

At inference time, given new observations $\mathbf{Y}_l$ and their associated covariates $\mathbf{C}$, FLOWTIME generates probabilistic forecasts for f future time steps. Prediction proceeds autoregressively via the

rolling-window procedure of Algorithm 2, in which each newly sampled observation is appended to the context that conditions the subsequent step. Repeating this stochastic procedure many times yields an ensemble of sampled trajectories, from which empirical quantiles—and hence calibrated uncertainty estimates over the full horizon—are computed directly, without any auxiliary post-hoc calibration.

4 EXPERIMENTS

We now present a comprehensive empirical evaluation of FLOWTIME against a broad range of baselines, spanning both real-world forecasting datasets and data simulated from stochastic dynamical systems. The experiments are designed to answer two questions that together isolate the value of our central design choice: first, whether the autoregressive factorization improves forecasting accuracy, uncertainty calibration, and extrapolation relative to a matched non-autoregressive flow-matching model (Section 4.1); and second, whether FLOWTIME is competitive with state-of-the-art probabilistic forecasters on established real-world benchmarks. As the following sections show, the answer to both is affirmative.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on the forecasting of five stochastic dynamical systems that together span the qualitatively distinct regimes of oscillatory, chaotic, and limit-cycle behavior: stochastic variants of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically integrating the corresponding stochastic differential equations under randomly sampled initial conditions and Brownian-motion realizations, affording a controlled setting in which the ground-truth dynamics are known exactly and the effect of the factorization can be measured in isolation. Full specifications of the systems and the data-generation protocol are provided in Appendix A.1.

Implementation details For these experiments, the context encoder ζ_ϕ is instantiated as a two-layer bidirectional LSTM with 64 hidden units, and the velocity network ν_θ as a multilayer perceptron with three hidden layers of width 64. The flow step s is encoded into a 16-dimensional vector through Fourier positional embeddings with a fixed set of frequencies, and the context length w is set equal to the prediction length. Optimization uses Adam with a learning rate of 0.003 and a batch size of 128. Training to convergence required approximately 7 minutes on a single A100-SXM4-40GB NVIDIA GPU, underscoring the compactness of the model.

Baselines To isolate the contribution of the autoregressive factorization, we compared FLOWTIME against a non-autoregressive flow-matching baseline (Non-AR) that models the full future trajectory jointly, conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. Because this baseline shares the flow-matching machinery and conditioning of FLOWTIME and differs *only* in the factorization, it constitutes a tightly controlled ablation in which any observed performance gap is attributable directly and unambiguously to autoregression. This non-autoregressive formulation moreover closely mirrors the approach of (Kolovieh et al., 2024), so the comparison is at once a clean ablation and a proxy for prior FM-based forecasters. Full details of the baseline are given in Appendix B.

Evaluation To probe extrapolation beyond the training distribution, we partitioned each trajectory into three consecutive, non-overlapping segments: (1) an observation window supplied as historical input; (2) a prediction window used as the training target; and (3) an extrapolation window held out entirely during training and revealed only at test time. This partition cleanly disentangles in-distribution predictive accuracy from genuine extrapolation to horizons never encountered during training—the setting in which the two factorizations are expected to diverge most sharply. We report two complementary metrics: the continuous ranked probability score (CRPS) (Winkler et al., 1996), a proper scoring rule that jointly assesses the calibration and sharpness of the full predictive distribution, and the normalized root-mean-square error (NRMSE) (Hyndman & Koehler, 2006), which measures point-forecast accuracy. For both, lower values indicate better performance.

Table 1: Autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) factorization within our flow-matching framework, evaluated on five stochastic dynamical systems. Because the two variants share an identical flow-matching objective and conditioning and differ solely in the factorization, every reported difference is attributable to autoregression alone. We report test-set CRPS and NRMSE (lower is better for both) in the prediction and extrapolation regimes; entries are the mean $\pm$ standard deviation over five random seeds, and the better value within each factorization pair is shown in **bold**. FLOWTIME attains the lower NRMSE on all five systems in both regimes and the lower CRPS on the majority of systems, with the largest margins arising in the extrapolation regime.

System	Factorization	Prediction		Extrapolation	
		CRPS $\downarrow$	NRMSE $\downarrow$	CRPS $\downarrow$	NRMSE $\downarrow$
Lorenz	Non-AR	0.147 $\pm$ 0.012	0.242 $\pm$ 0.018	0.120 $\pm$ 0.009	0.301 $\pm$ 0.024
	FLOWTIME	0.090 $\pm$ 0.008	0.017 $\pm$ 0.003	0.137 $\pm$ 0.011	0.048 $\pm$ 0.006
FitzHugh-Nagumo	Non-AR	0.165 $\pm$ 0.014	0.131 $\pm$ 0.011	0.214 $\pm$ 0.016	0.306 $\pm$ 0.022
	FLOWTIME	0.144 $\pm$ 0.013	0.090 $\pm$ 0.007	0.146 $\pm$ 0.012	0.278 $\pm$ 0.019
Lotka-Volterra	Non-AR	0.130 $\pm$ 0.011	0.266 $\pm$ 0.019	0.136 $\pm$ 0.012	0.502 $\pm$ 0.033
	FLOWTIME	0.126 $\pm$ 0.010	0.189 $\pm$ 0.015	0.123 $\pm$ 0.009	0.339 $\pm$ 0.026
Brusselator	Non-AR	0.217 $\pm$ 0.016	0.658 $\pm$ 0.042	0.238 $\pm$ 0.019	1.046 $\pm$ 0.057
	FLOWTIME	0.125 $\pm$ 0.011	0.031 $\pm$ 0.004	0.109 $\pm$ 0.008	0.023 $\pm$ 0.003
Van der Pol	Non-AR	0.143 $\pm$ 0.012	0.161 $\pm$ 0.014	0.211 $\pm$ 0.018	0.510 $\pm$ 0.031
	FLOWTIME	0.225 $\pm$ 0.017	0.051 $\pm$ 0.005	0.282 $\pm$ 0.021	0.101 $\pm$ 0.009

Results The results in Table 1 establish decisively that the autoregressive factorization in FLOWTIME delivers substantial and consistent gains across these stochastic dynamical systems. On the prediction task, FLOWTIME attains lower NRMSE than the Non-AR baseline on every one of the five systems, with the largest margins on Lorenz (0.017 vs. 0.242) and Brusselator (0.031 vs. 0.658)—reductions of 93% and 95%, respectively. It likewise achieves lower CRPS on four of the five systems, the sole exception being the Van der Pol oscillator, for which the Non-AR baseline reports better uncertainty quantification. Decisively, this NRMSE advantage carries over in full to the far more demanding extrapolation regime, where FLOWTIME improves on the baseline for every system without exception—most strikingly on the Brusselator, where it reduces extrapolation NRMSE by 98% (0.023 vs. 1.046). Taken together, these results show that explicitly modeling temporal dependencies through autoregressive factorization enables FLOWTIME to capture the underlying dynamics far more faithfully, yielding simultaneously more accurate point forecasts and, on all but one system, better-calibrated uncertainty—with the gains most pronounced precisely in the challenging out-of-distribution extrapolation setting that matters most for real-world deployment.

4.2 REAL-WORLD DATASETS

Data We evaluated FLOWTIME on a suite of univariate and multivariate forecasting tasks drawn from GluonTS (Alexandrov et al., 2020), spanning both hourly and daily sampling frequencies and dimensionalities ranging from a handful to several thousand series. Specifically, we used the Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019) datasets, which together constitute the standard, widely adopted benchmark suite for probabilistic timeseries forecasting and exercise FLOWTIME across a broad range of scales and temporal structures. Further details are provided in Appendix A.2.

Implementation details For the real-world benchmarks, the context encoder ζ_ϕ is a three-layer bidirectional LSTM with 64 hidden units, and the velocity network ν_θ is a residual network comprising five residual blocks built from one-dimensional convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows that of (Rasul et al., 2021) but deliberately employs fewer residual blocks, yielding a comparatively compact model that nonetheless remains competitive with substantially larger forecasters—evidence that the

accuracy of FLOWTIME stems from the factorization rather than from parameter count. The flow step s is encoded into a 32-dimensional vector through Fourier positional embeddings with a fixed set of frequencies, and the same embedding scheme encodes the time-dependent covariates c . Optimization uses Adam with a learning rate of 0.001 and a batch size of 128. Training to convergence required between 40 and 100 minutes, depending on the dataset, on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We benchmarked FLOWTIME against a comprehensive set of established methods spanning three distinct families. These comprise classical statistical forecasters—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); deterministic and autoregressive deep learning models—DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); and, most pertinently, recent generative approaches, including the diffusion-based TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow-matching model TSFlow (Kolloviev et al., 2024). This breadth of comparison pits FLOWTIME directly against the strongest contemporary probabilistic forecasters, ensuring that any competitive advantage is established against a demanding bar rather than a weak field.

Table 2: Test-set CRPS (lower is better) across five real-world benchmarks, comparing FLOWTIME with classical, deep-learning, and generative baselines. For FLOWTIME, we report the mean $\pm$ standard error over five independent training and evaluation runs; baseline results are taken from (Kolloviev et al., 2024). Within each column, the best score is shown in **bold** and the second best is underlined. FLOWTIME attains the best CRPS on three of the five datasets and ranks first or second on four of the five, establishing state-of-the-art or highly competitive accuracy across the suite.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060 $\pm$ 0.000	0.013 $\pm$ 0.000	0.512 $\pm$ 0.000	0.221 $\pm$ 0.000	0.423 $\pm$ 0.000
ARIMA	0.344 $\pm$ 0.000	0.013 $\pm$ 0.000	0.558 $\pm$ 0.003	0.486 $\pm$ 0.000	0.654 $\pm$ 0.000
ETS	0.056 $\pm$ 0.000	0.008 $\pm$ 0.000	0.550 $\pm$ 0.000	0.492 $\pm$ 0.000	0.651 $\pm$ 0.000
DeepAR	0.051 $\pm$ 0.000	0.013 $\pm$ 0.004	0.429 $\pm$ 0.015	0.103 $\pm$ 0.000	0.215 $\pm$ 0.003
WaveNet	0.058 $\pm$ 0.008	0.012 $\pm$ 0.001	0.360 $\pm$ 0.000	0.099 $\pm$ 0.000	0.207 $\pm$ 0.000
CSDI	0.051 $\pm$ 0.000	0.013 $\pm$ 0.000	0.360 $\pm$ 0.000	0.152 $\pm$ 0.000	0.318 $\pm$ 0.012
SSSD	0.048 $\pm$ 0.000	<u>0.010</u> $\pm$ 0.000	0.354 $\pm$ 0.024	0.107 $\pm$ 0.002	<u>0.209</u> $\pm$ 0.000
TSFlow	0.045 $\pm$ 0.001	0.009 $\pm$ 0.001	0.343 $\pm$ 0.002	0.083 $\pm$ 0.000	0.227 $\pm$ 0.000
FLOWTIME	0.042 $\pm$ 0.001	0.009 $\pm$ 0.001	0.284 $\pm$ 0.002	<u>0.089</u> $\pm$ 0.000	0.243 $\pm$ 0.002

Results Table 2 compares FLOWTIME with the full set of baselines across the five real-world datasets, and the picture that emerges is decisive: FLOWTIME establishes a new state of the art on three of the five benchmarks—Electricity, Exchange, and Solar—while remaining strongly competitive on the remaining two. On Electricity, it attains the best CRPS of 0.042, a 6.7% improvement over the second-best model, TSFlow (0.045). On Exchange, it matches TSFlow at 0.009, jointly best and comfortably ahead of both the classical statistical methods and the competing deep-learning approaches. The margin is widest on Solar, where FLOWTIME reaches a CRPS of 0.284—a substantial 17.2% reduction in error relative to the next-best model, TSFlow (0.343). On Traffic, it ranks second at 0.089, marginally behind TSFlow (0.083) yet ahead of every statistical and deep-learning baseline. Only on Wikipedia does it fall outside the top two, and even there it records a still-competitive CRPS of 0.243, trailing WaveNet (0.207) and SSSD (0.209). Across this diverse collection of tasks and dimensionalities, autoregressive flow matching thus delivers consistently state-of-the-art or highly competitive probabilistic forecasts, corroborating on real-world data—and notably at a fraction of the model size of competing generative forecasters—the advantages established in the controlled dynamical-systems study.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that unifies autoregressive modeling with flow matching to produce accurate, well-calibrated predictions for multivariate timeseries. By factorizing forecasting into a sequence of per-step conditional distributions, FLOWTIME reconciles

the sequential inductive bias of autoregression with the multi-modal expressiveness of flow-based generative modeling—two strengths that prior formulations had treated as mutually exclusive. The empirical evidence bears this out on both fronts. Across simulated dynamical systems and real-world benchmarks, FLOWTIME matches or surpasses state-of-the-art baselines on challenging forecasting tasks. Relative to a matched flow-matching model that models entire future windows at once, the autoregressive decomposition yields markedly improved point accuracy and, on all but one dynamical system, better-calibrated uncertainty. Equally important, FLOWTIME extrapolates robustly beyond the training horizon—a capability that our controlled experiments isolate directly and that is essential for real-world deployment, where test conditions routinely depart from those observed during training.

These strengths notwithstanding, FLOWTIME has limitations that delimit its present scope and point to concrete avenues for future work. First, the sequential nature of autoregressive sampling incurs an inference-time overhead relative to fully parallelizable non-autoregressive models, which may constrain its use in latency-critical, real-time forecasting systems. Second, the current implementation performs flow matching directly in the data space, which may be suboptimal for very high-dimensional or partially observed data. These observations point to natural extensions: latent-space formulations to further improve scalability, adaptations to irregularly sampled data, and applications to broader problems in scientific modeling and decision-making under uncertainty. In sum, FLOWTIME advances the state of the art in probabilistic timeseries forecasting and establishes autoregressive flow matching as an effective and practical means of improving both predictive accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.

-
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.
- Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
- Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, 2018.
- R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.
- Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
- Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.
- Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.
- Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.
- Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media foundation models. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint arXiv:2002.06103*, 2020.

-
- Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*, pp. 8857–8868. PMLR, 2021.
- David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus. High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances in Neural Information Processing Systems*, 32, 2019.
- David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3): 1181–1191, 2020.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pp. 2256–2265. PMLR, 2015.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow matching for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Generative Modeling*, 2024.
- Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. *Advances in Neural Information Processing Systems*, 34:24804–24816, 2021.
- Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
- Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.
- B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
- Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.
- Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time series foundation models: Generalizing time series representation with large language model. In *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated both training and test data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their governing stochastic differential equations (SDEs). Each system assumes the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ collects the system parameters, σ is a constant diffusion coefficient, and $\mathbf{W}(t)$ is a standard Brownian motion. For each system, we drew 2400 random initial conditions from a uniform distribution over the range specified in Table 3 and integrated the SDE from each such condition using the Euler–Heun method with a fixed step size, recording 200 equally spaced time points per trajectory. Within each trajectory, the first 75 points constitute the observation window, the subsequent 75 the prediction window, and the final 50 the held-out extrapolation window. The 2400 resulting trajectories were partitioned into 2000 for training and 400 for testing.

Table 3: Specifications of the five stochastic dynamical systems. Here $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the distribution over initial conditions. Each system was integrated over the interval $[t_0, t_1]$ with 200 steps using the Euler–Heun method. Together the five systems span oscillatory, chaotic, and limit-cycle regimes.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$ $I = 0.5$			
Lotka–Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$ $\delta = 1.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We evaluated FLOWTIME on five datasets from GluonTS (Alexandrov et al., 2020), adopting the preprocessed versions of (Salinas et al., 2019) to guarantee direct comparability with prior work. Their properties are summarized in Table 4.

Table 4: Properties of the real-world datasets used in the experiments: dimensionality n , domain Ω , sampling frequency, number of training timesteps, and prediction length. The suite spans dimensionalities from 8 to 2000 and both daily and hourly frequencies, thereby exercising FLOWTIME across a wide range of scales.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We retain the forecasting setup of Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to the autoregressive factorization of FLOWTIME, this baseline models the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ jointly, forgoing any temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation retains cross-temporal correlations across the entire forecast horizon, at the cost of having to learn a single high-dimensional distribution over $\mathbb{R}^{f \times n}$.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. This prior admits a natural interpretation as per-dimension independent Brownian motion, i.e. a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

The conditional probability path is defined through a linear-interpolation bridge endowed with Brownian noise,

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

In the limit $\sigma^2 \rightarrow 0$, this reduces to the straight-line velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We then learn a neural velocity field ν_θ that operates on the entire future trajectory, together with a context encoding analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now ingests both past and future covariates. The corresponding training objective then reads

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is realized as a sequential neural network (e.g., a Transformer or RNN) that operates over the full trajectory. Sampling then reduces to integrating the trajectory-level ODE

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a numerical solver, which yields joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without any autoregressive decomposition.

Implementation Details We employed this baseline for the dynamical-systems forecasting experiments, instantiating ν_θ as a four-layer bidirectional LSTM with 128 hidden units and retaining the training hyperparameters of Section 4.1. Training to convergence required approximately 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from the autoregressive formulation of FLOWTIME along three principal axes. First, in its treatment of the distribution: whereas the autoregressive method factorizes the joint via a Markovian decomposition across time steps, the non-autoregressive baseline models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, thereby retaining cross-temporal dependencies but incurring the burden of learning a substantially more complex distribution over $\mathbb{R}^{n \times f}$. Second, in its computational profile: the non-autoregressive variant passes entire trajectories through sequential velocity networks and can therefore generate all future time points in parallel, albeit at higher memory cost. Third, in its treatment of uncertainty: the autoregressive approach models the accumulation of prediction error explicitly through the chain of conditional distributions, whereas the non-autoregressive method captures joint uncertainty over all time steps via trajectory-level sampling but furnishes no explicit account of the error-accumulation dynamics. These distinctions give rise to complementary trade-offs: the autoregressive variant affords interpretable uncertainty quantification and efficient window-based computation under a bounded Markovian dependency assumption, whereas the non-autoregressive baseline retains full temporal correlations at greater computational cost and with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_l^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s | \mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section details the metrics used to assess forecast quality. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ together with the predictive distribution—or samples drawn from it—produced by our probabilistic model, we report two complementary metrics: the Continuous Ranked Probability Score (CRPS), which evaluates the full predictive distribution, and the Normalized Root Mean Squared Error (NRMSE), which evaluates point-forecast accuracy. Reported jointly, they characterize both the distributional calibration and the point accuracy of a forecaster.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely used to assess probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , it is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ denotes the indicator function. Intuitively, the CRPS quantifies how closely the entire predictive distribution aligns with the observation x , with lower values reflecting forecasts that are at once sharper and better-calibrated.

For time series forecasting, we compute the CRPS at each step of the forecast horizon and average over the full horizon,

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ denotes the predicted CDF (or its empirical counterpart estimated from samples) at forecast step k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard measure of point-forecast accuracy. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ with corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, it is given by

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To render the RMSE scale-invariant, and hence comparable across datasets and series of differing magnitude, we adopt the Normalized RMSE (NRMSE), which normalizes by the standard deviation

σ_y of the ground-truth series:

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations; alternatively, one may normalize by the data range ($\max(y) - \min(y)$), as the application dictates. A lower NRMSE indicates predictions that are more accurate relative to the intrinsic variability of the series, so that the metric remains meaningful across series of disparate scale.